4.2 Subclasses & Descendants

4.2.1 `subclasses`

The `subclasses` method returns the subclasses of the receiver:

```
class C; end
C.subclasses # => []

class B < C; end
C.subclasses # => [B]

class A < B; end
C.subclasses # => [B]

class D < C; end
C.subclasses # => [B, D]
```

The order in which these classes are returned is unspecified.

Defined in `active_support/core_ext/class/subclasses.rb`.

4.2.2 `descendants`

The `descendants` method returns all classes that are < than its receiver:

```
class C; end
C.descendants # => []

class B < C; end
C.descendants # => [B]

class A < B; end
C.descendants # => [B, A]

class D < C; end
C.descendants # => [B, A, D]
```

The order in which these classes are returned is unspecified.

Defined in `active_support/core_ext/class/subclasses.rb`.

5 Extensions to `String`

5.1 Output Safety

5.1.1 Motivation

Inserting data into HTML templates needs extra care. For example, you can't just interpolate `@review.title` verbatim into an HTML page. For one thing, if the review title is "Flanagan & Matz rules!" the output won't be well-formed because an ampersand has to be escaped as "&". What's more, depending on the application, that may be a big security hole because users can inject malicious HTML setting a hand-crafted review title. Check out the section about cross-site scripting in the Security guide for further information about the risks.

5.1.2 Safe Strings

Active Support has the concept of *(html) safe* strings. A safe string is one that is marked as being insertable into HTML as is. It is trusted, no matter whether it has been escaped or not.

Strings are considered to be *unsafe* by default:

```
"".html_safe? # => false
```

You can obtain a safe string from a given one with the `html_safe` method: